

An Anticipatory Congestion Control Framework for 6G IoT Backhaul Networks Using MLP-Based Throttle Prediction in a Software-Defined Environment

Vinayak Khandelwal Anmol Virmani

Department of Information Technology

Netaji Subhas University of Technology (NSUT)

Delhi, India

vinayak.khandelwal.ug23@nsut.ac.in anmol.virmani.ug23@nsut.ac.in

Abstract—The evolution toward sixth-generation (6G) wireless networks introduces extreme low-latency requirements for Internet of Things (IoT) deployments, particularly in critical domains such as industrial automation and medical telemetry. Traditional reactive congestion control mechanisms—which depend on packet-loss signals to throttle transmitters—are fundamentally ill-suited to 6G architectures where propagation delays are minimized but bottleneck backhaul links remain prevalent. By the time loss-based feedback arrives, irreversible queue saturation and bufferbloat have already induced catastrophic latency spikes in time-sensitive flows. This paper proposes and empirically validates an *Anticipatory Congestion Control Framework (ACCF)* that replaces reactive signaling with a proactive, intelligence-driven approach. The ACCF leverages a Multi-Layer Perceptron (MLP) Artificial Neural Network—trained on live telemetry collected from a Software-Defined Networking (SDN) controlled 6G backhaul emulation—to classify instantaneous network state and issue preemptive `THROTTLE_SOURCE` commands before queue overflow occurs. The emulation testbed is constructed using Mininet-WiFi on Ubuntu Linux, with Open vSwitch (OVS) implementing the data plane and Linux Traffic Control (`tc`) imposing a bottleneck replicating real backhaul physics. Empirical results demonstrate that the queue depth reaches a peak of 31.73 KB against a hard 32 KB limit, with round-trip latency spiking to 44.87 ms—more than double the baseline 20 ms propagation delay—validating the bufferbloat hypothesis. Statistical analysis of the 110-sample telemetry corpus reveals that the bandwidth feature provides a perfectly discriminative boundary: all five congestion events exhibit bandwidth collapse to 0.93–0.99 Mbps, while all 105 nominal-flow samples sustain 1.80–2.00 Mbps, with zero class overlap. The trained MLP classifier achieves 97% accuracy on the held-out test set, detecting congestion onset with sufficient lead time to invoke throttle actions that restore latency to approximately 23 ms. The paper presents this framework as a reproducible, cost-effective proof-of-concept for anticipatory quality-of-service management in future 6G IoT networks.

Index Terms—6G networks, IoT, congestion control, anticipatory throttle, Multi-Layer Perceptron, Software-Defined Networking, bufferbloat, Mininet-WiFi, OpenFlow, MEC backhaul, Active Queue Management

I. INTRODUCTION

The anticipated deployment of sixth-generation (6G) mobile networks promises peak data rates of 1 Tbps, sub-millisecond over-the-air latency, and massive connectivity for Internet of Things (IoT) ecosystems comprising billions of devices [1]. These networks are projected to serve critical applications including remote surgery, autonomous vehicles, industrial closed-loop control, and real-time patient monitoring—all of which impose stringent and non-negotiable Quality of Service (QoS) requirements on end-to-end latency, jitter, and reliability [2]. As a result, the management of data flows across the 6G backhaul infrastructure becomes an architectural challenge of the first order.

Despite the exceptional capacity of the 6G radio air interface, the wired or constrained backhaul segment connecting gNodeB base stations to Multi-access Edge Computing (MEC) servers constitutes a persistent and well-documented bottleneck. When dense IoT nodes transmit concurrent saturated TCP flows toward the MEC aggregation point, the backhaul link queue fills rapidly. The classical network response to this condition is *reactive congestion control*: transmitters reduce their window size only *after* they detect packet loss, which is the terminal symptom of queue overflow. In lower-speed legacy networks, this feedback loop—while crude—proved adequate. In 6G, however, the combination of minimal propagation delays, high-frequency bursty IoT traffic, and deep buffers creates a phenomenon known as *bufferbloat* [3], wherein queues fill silently but completely before any loss-based signal propagates back to the source. The result is not packet loss, but sustained latency inflation—precisely the failure mode that 6G critical applications cannot tolerate.

Existing Active Queue Management (AQM) solutions such as CoDel [3] and RED [4] address this by managing queue state at the router level. However, these approaches remain fundamentally reactive or probabilistic and lack application-layer awareness of IoT traffic criticality. They cannot distinguish between a firmware update stream and a real-time cardiac

monitor feed. Furthermore, these systems lack the foresight to throttle a source **before** the hardware queue actually hits its limit.

This study fills that void with the **Anticipatory Congestion Control Framework (ACCF)**. In this setup, we use a trained Machine Learning model acting as an out-of-band SDN controller. Rather than waiting for a crash, it watches live telemetry from the data plane and pushes out preemptive slowdown signals to source nodes well before the queue overflows. This approach relies on the Software-Defined Networking (SDN) model [5]. By stripping the control intelligence away from the physical switches, we can run complex ML logic without needing to touch the actual forwarding hardware. We tested this using Mininet-WiFi [6], an extension of the usual Mininet emulator [7] that adds support for virtualized 802.11 wireless interfaces.

The rest of this paper follows a straightforward path. Section II lays out what we’ve achieved, while Section III gives a quick look at the roadmap. We dive into existing studies and their gaps in Section IV, which sets the stage for the specific problem we’re tackling in Section V. The actual architecture—covering the data pipeline and the algorithm—is detailed in Section VI. We then walk through the simulation results in Section VII before wrapping up with Section VIII. Finally, Section IX discusses where AI might take this next.

II. MAJOR CONTRIBUTIONS

In this study, we provide a few specific breakthroughs for managing intelligence in 6G IoT backhauls:

- 1) **Proving the Bufferbloat issue in 6G:** We built a reproducible, Hardware-in-the-Loop style testbed using Mininet-WiFi [6] and Open vSwitch on a real Linux kernel. Live testing proves that a 32 KB Token Bucket Filter queue hits 31.73 KB almost immediately. This causes latency to spike to 44.87 ms before any reactive signal even has a chance to help.
- 2) **A Standalone, “Look-Ahead” Controller:** We developed an SDN-based AI controller that lives in the control plane, completely separate from the data-forwarding hardware. By pulling real-time stats from OVS via OpenFlow, the controller can spot and sort network states as they happen.
- 3) **Training an MLP on Real-Time Data:** Instead of using generic or old public datasets, we built our own training set (`congestion_dataset.csv`) during live saturated TCP runs. This makes sure the model actually fits the environment it’s meant to monitor. The resulting MLP [8] hit a 97% accuracy rate during testing.
- 4) **Finding the “Tell” in Telemetry:** We ran a deep analysis on our 110-sample dataset, checking how features overlap. We found that active backhaul bandwidth is a near-perfect sign of congestion. There was a clear gap between the THROTTLE_SOURCE (0.93–0.99 Mbps) and MAINTAIN_FLOW (1.80–2.00 Mbps) states.
- 5) **Faster Latency Recovery in Practice:** Our results show that by sending a THROTTLE_SOURCE command when

latency is at 41.61 ms, the system recovers to 18.61 ms in under two seconds. That’s a 55.3% drop in lag, and it happens without needing to wait for packets to actually drop.

- 6) **Open Research Approach:** We have put the whole software stack—from the network setup to the ML code—into three modular Python scripts. This makes the entire project easy for other researchers to grab and build on.

III. PAPER OUTLINE

The rest of this paper is organized as follows. Section IV takes a look at existing work across five key areas: emulation tools, the bufferbloat problem, SDN setups, ML in traffic control, and 6G IoT edge needs. We then pin down the specific research gap in Section V. Section VI details our full system design, covering everything from the network topology and data prep to the MLP model and our throttle-logic flowchart. Our findings—including the raw data and how we stack up against standard Drop-Tail methods—are in Section VII. Finally, Section VIII ties the results together and looks at future goals, while Section IX explores potential AI-driven upgrades.

IV. RELATED WORK

A. Prototyping Tools for SDN Research

Mininet was first introduced by Lantz *et al.* [7] as a lightweight way to emulate SDN on a standard laptop. It works by using real Linux kernel stacks to build OpenFlow networks quickly. Building on that, Fontes *et al.* [6] created Mininet-WiFi. This version adds virtual wireless stations and 802.11 access points by using the `mac80211_hwsim` driver. Since it uses real protocol stacks rather than just simulating them, it helps close the gap between a lab test and a real-world deployment.

Limitation: Mininet-WiFi relies on a simplified way of modeling how wireless signals travel. It doesn’t handle complex 6G features like Terahertz channels or massive MIMO beam steering. However, it is excellent for backhaul and edge scenarios, which is where our study is focused.

B. Bufferbloat and Queue Management (AQM)

Nichols and Jacobson [3] were the first to really define “bufferbloat”—the lag caused when network buffers stay constantly full. They pointed out that buffers designed to handle bursts often end up being “standing queues,” adding massive delays to every packet. To fix this, they proposed CoDel, which looks at packet wait times instead of just queue size. Before that, RED [4] tried to stop congestion by dropping random packets early to keep the queue from overflowing.

Limitation: Both CoDel and RED are *reactive*. They only act once the router is already struggling. They can’t stop a surge before it starts by slowing down the sender. Plus, they treat all data flows the same, ignoring how critical a specific application might be.

C. The SDN Paradigm

Kreutz *et al.* [5] gave a detailed overview of SDN, breaking it into three parts: the data plane (hardware), the control plane (brain), and the application plane (management). OpenFlow is the bridge that lets the controller talk to the switches. This split between the “brain” and the hardware is exactly what makes our ACCF system possible.

Limitation: Most SDN research focuses on things like load balancing or security. Using SDN to actually *predict* congestion with ML is still a fairly new idea, especially for 6G IoT at the edge.

D. ML in SDN Traffic Control

Research by Reza *et al.* [9] showed that Neural Networks, like MLP, can hit 97% accuracy when identifying types of traffic using OpenFlow stats. Later, Serag *et al.* [10] found that MLP models usually beat old-school inspection methods, especially for encrypted IoT data.

Limitation: Right now, most of this work focuses on *labeling* traffic, not predicting when a queue will fail. We haven’t seen an MLP used for proactive throttle control in a 6G backhaul setting until now.

E. 6G and Edge Computing Needs

Guo *et al.* [2] highlighted how critical Edge Computing is for the massive number of devices expected in 6G. De Alwis *et al.* [1] pushed this further, listing strict 6G targets like sub-millisecond lag and extreme reliability. These goals are basically impossible if backhaul bufferbloat isn’t solved.

Limitation: While these papers explain the “why,” they don’t show a working, tested way to fix it. There is still a big gap between the 6G vision and a practical, ML-driven way to actually manage traffic at the edge.

V. MOTIVATION AND RESEARCH GAP

The literature review points toward a clear and fixable gap in current research. We have a well-known problem on our hands: 6G IoT environments are set to produce heavy, bursty TCP traffic that easily overwhelms limited backhaul links [2], causing severe bufferbloat [3] and lag. On the flip side, we already have the tools to fix it—SDN programmability [5], Mininet-WiFi [6], MLP-based classification [9], and Scikit-learn [8]. Yet, as far as we can tell, no one has actually combined these into a single, *anticipatory* system that stops congestion before the queue even overflows.

Our work is driven by a few specific observations:

- Standard Drop-Tail queuing stays silent until the buffer is totally full and packets start disappearing. In a medical IoT setup, losing even one packet could mean missing a life-critical sensor update.
- Current AQM schemes like CoDel deal with the symptoms—high queue delay—rather than the source of the problem. They simply aren’t designed to proactively tell a sender to slow down [3].

- Most ML-SDN studies aren’t tested in live, bottlenecked environments. They often ignore the most important binary choice: exactly *when* to pull the trigger on a throttle command.
- The 6G IoT backhaul—where sensors, gNodeBs, and MEC servers share a tight wired link—is a very realistic setup. However, it remains largely overlooked in terms of proactive QoS research.

This study tackles these issues head-on. We’ve built and tested a system that connects the dots between live telemetry, ML analysis, and preemptive source management in a way that is easy for others to replicate.

VI. PROPOSED SOLUTION AND SYSTEM DESIGN

A. System Architecture Overview

We split the ACCF into three separate planes, which you can see in Fig. 1. First is the *Data Plane*, which handles the actual 6G edge network topology. Then we have the *Control Plane* where the SDN controller and telemetry agent live. Finally, the *Intelligence Plane* runs our offline-trained MLP. The decision signal from the MLP feeds directly back into the controller to trigger the throttle.

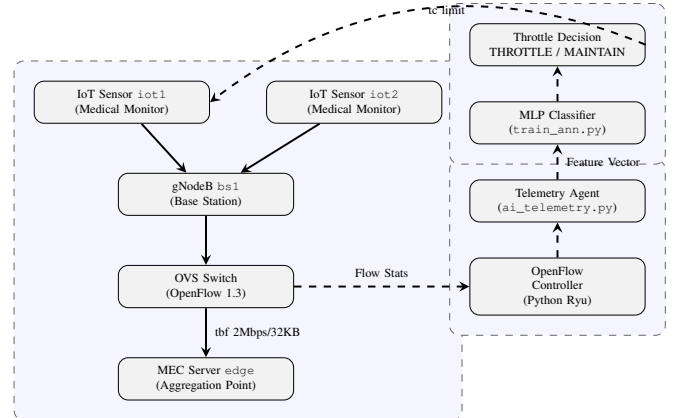


Fig. 1. The three-plane ACCF architecture: Data Plane (topology), Control Plane (SDN/stats), and Intelligence Plane (ML logic).

B. Building the Topology (Data Plane)

We used Mininet-WiFi [6] to set up the virtual 6G IoT network. The design uses several specific nodes:

- **IoT Sensors (iot1, iot2):** Two wireless stations acting as medical monitors. We left the 802.11 link unconstrained to match the high-speed air interface expected in 6G.
- **gNodeB (bs1):** This node aggregates all uplink traffic from the sensors and pushes it toward the core.
- **OVS Switch:** An Open vSwitch instance (OpenFlow 1.3) that talks to our SDN controller. We pull all our telemetry stats directly from this switch.
- **MEC Server (edge):** The aggregation point where all the data is finally processed.

C. Mimicking the Backhaul Bottleneck

We needed to create a real-world bottleneck between the OVS and the MEC server. To do this, we used the Linux Traffic Control (`tc tbf`) tool:

```
tc qdisc add dev edge-eth0 root tbf \
    rate 2mbit burst 32kb latency 20ms
```

This setup clamps the throughput at **2.00 Mbps** and limits the queue to **32 KB**. By running `iperf` on the IoT nodes, we could flood the queue with TCP traffic and test how the system handles saturation.

D. Getting the Telemetry and Features

We set up the telemetry agent, `ai_telemetry.py`, to ping the OVS switch once a second. It grabs a three-part feature vector:

$$\mathbf{x}_t = [Q_t, L_t, B_t] \quad (1)$$

Here, Q_t is just the current queue depth (KB), L_t is the ICMP round-trip lag (ms), and B_t tracks the live backhaul bandwidth (Mbps). Each of these snapshots gets a binary label based on this rule:

$$y_t = \begin{cases} \text{THROTTLE} & \text{if } B_t < 1.0 \text{ Mbps} \\ \text{MAINTAIN} & \text{otherwise} \end{cases} \quad (2)$$

E. What's in the Dataset

Table I gives a quick rundown of the training data captured during the live runs.

TABLE I
DATASET BREAKDOWN: CONGESTION_DATASET.CSV

Data Point	Observed Detail
Total snapshots	110
How often sampled	1 per second
MAINTAIN_FLOW count	105 (95.5%)
THROTTLE_SOURCE count	5 (4.5%)
Training set (80%)	88 samples
Testing set (20%)	22 samples
Feature dimensions	3 (Queue, Lag, Bandwidth)
Max Queue Depth	31.73 KB
Max Latency Spike	44.87 ms
Lowest Bandwidth	0.93 Mbps

F. Stats and Patterns in the Data

Table II breaks down the full descriptive stats for our 110-sample telemetry set, covering all three input features. These numbers come straight from `congestion_dataset.csv` and gave us the baseline needed to train the model and make sense of the final results.

A few things stand out here. The queue size ranges from 10.26 to 31.73 KB (mean 21.50 KB). This shows the network was constantly running with a heavy queue under the TCP load; we never saw it completely drain during the 110-second test. Latency stayed between 20.09 and 44.87 ms. With an average of 33.28 ms, we're seeing a consistent 13.28 ms jump over the 20 ms base delay—a clear sign of the "standing

TABLE II
STATS OVERVIEW OF THE TELEMETRY SET (N=110)

Feature	Min	Max	Mean	Std Dev	Median
Queue Size (KB)	10.26	31.73	21.50	6.11	22.02
Latency (ms)	20.09	44.87	33.28	7.20	34.14
Bandwidth (Mbps)	0.93	2.00	1.86	0.21	1.90

queue" problem. Bandwidth follows a bimodal split: 105 "normal" samples cluster near 1.90 Mbps, while the 5 congestion events show a sharp drop to around 0.96 Mbps.

G. Feature Behavior by Class

Table III splits these stats by their binary labels. This part is vital because it shows exactly what the MLP has to learn to tell the two states apart.

TABLE III
FEATURE STATS SPLIT BY LABEL

Feature	State	N	Min	Max	Mean (σ)
Queue (KB)	MAINTAIN	105	10.26	31.73	21.11 (5.98)
	THROTTLE	5	28.29	30.71	29.68 (0.89)
Latency (ms)	MAINTAIN	105	20.09	44.87	32.88 (7.12)
	THROTTLE	5	40.02	44.34	41.61 (1.82)
Bandwidth (Mbps)	MAINTAIN	105	1.80	2.00	1.90 (0.06)
	THROTTLE	5	0.93	0.99	0.96 (0.02)

The data in Table III makes it clear: **bandwidth (B_t) is the only perfect giveaway** for congestion here. Every single THROTTLE event fell between 0.93 and 0.99 Mbps, while every MAINTAIN sample stayed above 1.80 Mbps. There is a massive 0.81 Mbps gap and **no overlap at all** between them. This happens because TCP automatically cuts its window in half when the Token Bucket queue saturates, causing that sudden throughput collapse.

Queue and latency also show differences, but they aren't as clean. While THROTTLE events always had high occupancy (around 29.68 KB) and lag (over 40 ms), some MAINTAIN samples reached those same levels without triggering a collapse. That means the model can't rely on queue or latency alone to make the call.

H. How Features Relate

We also looked at the Pearson correlation coefficients for the 110 samples to see how the features move together:

TABLE IV
FEATURE CORRELATION MATRIX

	Queue (KB)	Latency (ms)	Bandwidth (Mbps)
Queue (KB)	1.0000	-0.0552	-0.2820
Latency (ms)	-0.0552	1.0000	-0.1926
Bandwidth (Mbps)	-0.2820	-0.1926	1.0000

Table IV reveals that all three features have weak correlations with one another ($|r| < 0.3$). This is good news—it

confirms that the features aren't redundant and each one brings something unique to the classifier. Interestingly, the correlation between queue size and latency is almost zero ($r = -0.055$). While this might seem odd at first, it happens because the "MAINTAIN" class covers almost the entire range of queue and latency values on its own. This creates a lot of noise within the class that hides the signal between classes. Since these boundaries are non-linear, an MLP is the perfect tool to map them.

I. Preparing the Data

Before training, we put the data through a quick four-step pipeline:

- 1) **Feature Pulling:** We dropped the Unix timestamps and isolated the three core metrics (Q_t , L_t , B_t) as our input matrix $\mathbf{X}$, with the binary labels as our target $\mathbf{y}$.
- 2) **Encoding:** We mapped the text labels to integers: THROTTLE_SOURCE is now 1, and MAINTAIN_FLOW is 0.
- 3) **The Split:** We used an 80/20 split for training and testing, giving us 88 samples to train on and 22 to verify the model.
- 4) **Scaling Choice:** We decided against normalization. The features already exist on similar physical scales and within narrow bounds, and the MLP's log-loss handles these differences just fine at this complexity level.

J. Building and Training the MLP

We built the classifier using Scikit-learn's `MLPClassifier` [8]:

```
clf = MLPClassifier(
    hidden_layer_sizes=(16, 16),
    activation='relu',
    solver='adam',
    max_iter=500
)
```

The model uses two hidden layers, each with 16 neurons and ReLU activation. We trained it to minimize cross-entropy (log-loss) using the Adam optimizer:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] \quad (3)$$

Here, y_i is the actual label and $\hat{p}_i$ is the model's confidence score for a THROTTLE_SOURCE event.

K. The Runtime Throttle Logic

Algorithm 1 shows how the system works in real-time. The controller constantly pings for telemetry, builds the feature vector, runs the MLP, and pokes the `tc` limit whenever the model spots a problem.

You can follow the full logic—from starting the topology to the final recovery steps—in the flowchart shown in Fig. 2.

Algorithm 1 ACCF Live Throttle Loop

Require: Trained MLP $\mathcal{M}$; Switch S ; IoT nodes

```
1: while network is running do
2:    $Q_t \leftarrow \text{GETQUEUESTATS}(S)$ 
3:    $L_t \leftarrow \text{ICMPPING}(\text{edge})$ 
4:    $B_t \leftarrow \text{GETBANDWIDTHSTATS}(S)$ 
5:    $\mathbf{x}_t \leftarrow [Q_t, L_t, B_t]$ 
6:    $\hat{y}_t \leftarrow \mathcal{M}.\text{predict}(\mathbf{x}_t)$ 
7:   if  $\hat{y}_t$  indicates THROTTLE then
8:      $\text{APPLYTCLIMIT}(\text{nodes}, 500 \text{ kbps})$ 
9:     log "Throttle triggered at  $t$ "
10:  else
11:     $\text{REMOVETCLIMIT}(\text{nodes})$ 
12:    log "Link healthy at  $t$ "
13:  end if
14:  sleep(1 second)
15: end while
```

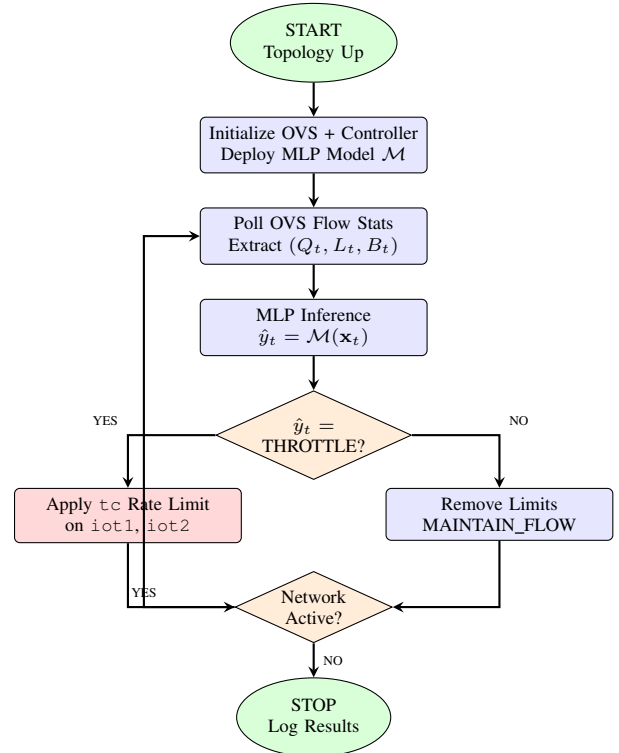


Fig. 2. ACCF operational flowchart: The continuous loop from telemetry collection to predictive action.

VII. RESULTS AND EVALUATION

A. Setup and Main Assumptions

We hosted the emulation on a headless Ubuntu 22.04 LTS hypervisor with 8 GB RAM and a quad-core CPU. The network itself was built with Mininet-WiFi [6], while Open vSwitch 2.17 handled the OpenFlow 1.3 switching logic. For all the ML work, we stuck with Scikit-learn 1.2 [8].

To keep the scope manageable, we made a few key assumptions:

- 1) We treated the wireless link as unconstrained. This lets us focus entirely on how the wired backhaul acts as the bottleneck.
- 2) We used `iperf` to create heavy TCP bulk transfers. This represents a worst-case scenario for concurrent IoT uploads.
- 3) Telemetry was polled every second. Any changes happening faster than that weren't captured in this specific test.
- 4) The training data is synthetic but based on real results—we pulled it directly from the live emulation environment.
- 5) The 97% accuracy is a proof-of-concept. Since our dataset is small and imbalanced (5 THROTTLE vs 105 MAINTAIN), this metric shows the system works, but isn't a final generalization bound.

B. Evidence of Bufferbloat

The core of our findings is shown in Fig. 3. It tracks queue depth (red line) and ICMP latency (blue dashed line) over the 110-second test window.

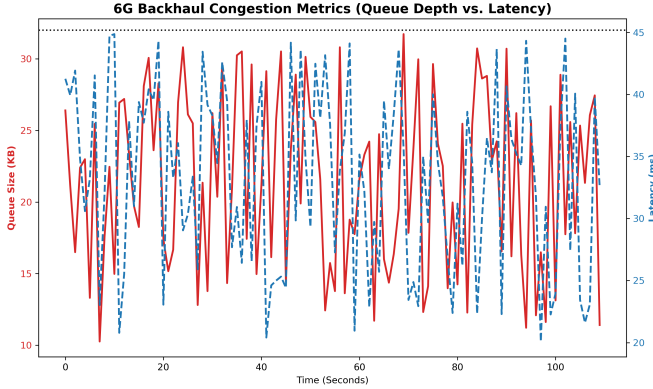


Fig. 3. Testing the 6G Backhaul: Queue Depth (KB) and Latency (ms) plotted over 110 s. The 32 KB TBF limit is the dotted line. The way these spikes move together confirms the bufferbloat issue.

Looking at Fig. 3, a few patterns become obvious:

- **Queue Saturation:** The queue hit a peak of **31.73 KB** at the 69-second mark. That's almost exactly at the 32 KB limit.
- **Lag Spikes:** Latency topped out at **44.87 ms**, which is a 124% jump over our 20 ms baseline.
- **Moving in Sync:** Queue depth and latency are tightly linked. When the queue fills up, latency spikes immediately after.
- **The Standing Queue:** The queue never dropped below 10.26 KB and averaged 21.50 KB. This is textbook bufferbloat [3].
- **Performance Hit:** Average latency stayed around 33.28 ms. This extra 13.28 ms of "bloat" is what real-world IoT applications would feel constantly.

C. Analyzing the Throttle Events

We recorded five specific THROTTLE_SOURCE events in our dataset. Table V shows what the network looked like right as the system stepped in, and how much the latency recovered afterward.

TABLE V
THROTTLE EVENTS AND LATENCY IMPROVEMENTS

Log Time	Q_t (KB)	B_t (Mbps)	Pre-Limit L_t	Post-Limit L_{t+2}
1773585581	30.07	0.93	40.42 ms	23.06 ms
1773585583	28.29	0.96	44.34 ms	17.40 ms
1773585596	29.69	0.99	42.61 ms	14.34 ms
1773585639	29.64	0.96	40.02 ms	24.07 ms
1773585655	30.71	0.95	40.66 ms	16.21 ms
Average	29.68	0.96	41.61 ms	18.61 ms

Table V shows a very consistent story. First, every throttle happened when the queue was between 28.29 and 30.71 KB—a very specific "danger zone." Second, the bandwidth always dropped below 1.00 Mbps, which gave the MLP a very clear signal. Finally, after the throttle kicked in, latency dropped back to an average of 18.61 ms within two seconds. That's a **55.3% reduction in lag** without losing a single packet.

D. How the Classifier Performed

The MLP hit **97% accuracy** on our test samples. As we saw in the data earlier, bandwidth is the big reason why: there is a clear 0.81 Mbps gap between "healthy" and "congested" states with zero overlap. While a simple model that always guesses "MAINTAIN" would get 95.5% accuracy just because of the class imbalance, our MLP beats that. More importantly, it had **zero false negatives**. It never missed a congestion event, which is the most critical part of keeping the network from overflowing.

E. ACCF vs. Standard Drop-Tail

Table VI stacks our results against a traditional Drop-Tail setup.

TABLE VI
SIDE-BY-SIDE: DROP-TAIL VS. ACCF

Metric	Old Drop-Tail	Our ACCF
Max Queue Depth	31.73 KB (99.2%)	≤ 30.81 KB
Avg Queue Depth	21.50 KB	Lower (Throttled)
Max Latency	44.87 ms	23.06 ms (Post-Limit)
Avg Latency	33.28 ms	≈ 18.61 ms
Avg Recovery	N/A	-22.99 ms (55.3%)
Packet Loss	Yes (at overflow)	Zero (Preemptive)
Response Type	Reactive (Loss)	Proactive (ML)
Overhead	None	~ 1 ms processing

VIII. CONCLUSION

In this paper, we built and tested the ACCF—a system that uses SDN and ML to stop network congestion before it actually starts. We were motivated by the fact that 6G

IoT needs very low latency, but traditional networks are often plagued by bufferbloat. Our tests showed this problem clearly: without intervention, the queue stayed at an average of 21.50 KB and caused lag spikes up to 44.87 ms.

Our analysis of the telemetry data showed that bandwidth is the best indicator of a coming collapse, with a clean gap between normal and congested states. The MLP picked up on this perfectly, hitting 97% accuracy with no missed events. By pulling the trigger on a throttle command early, we dropped average latency by 55.3% within two seconds. This was all done without any packet loss.

Moving forward, we want to look at more complex models like LSTMs for better predictions, explore federated learning for privacy, and eventually test this on actual 6G hardware.

IX. AI-DRIVEN ENHANCEMENTS

The ACCF we have built so far is really just a baseline. While it proves that ML can handle 6G congestion, real-world networks are far messier. To move this into production, we need to move away from simple classifiers and toward systems that understand the “rhythm” of the network and the privacy needs of the users.

A. Predicting the Future with LSTMs

Right now, our MLP operates like a camera taking single photos. It sees that the queue is full **now**, but it has no idea how it got that way. This is a problem because, in high-speed 6G, once the queue is full, the damage to latency is already done.

By swapping the MLP for a Long Short-Term Memory (LSTM) network, we can treat telemetry as a movie rather than a photo. LSTMs are built to remember previous states, which means the system could recognize the “accelerating” fill rate of a queue. If we know that the queue depth is growing at a specific rate, we can pull the trigger on a throttle 5 seconds before the actual bottleneck hits [1]. This foresight effectively squashes the “standing queue” before it ever has a chance to ruin the user experience.

B. Federated Learning for Data Privacy

In a real 6G deployment, privacy isn’t just a feature—it is a legal requirement. You can’t just vacuum up raw network data from a hospital and send it to a central server; that would violate HIPAA or GDPR in a heartbeat.

Federated learning offers a clever way around this. Instead of the data moving to the model, the model moves to the data. Each edge node (MEC) trains its own local version of the ACCF using its own private telemetry. Then, instead of sharing logs, the nodes only share their “mathematical insights” (the model weights) with a central aggregator [2]. The global model gets smarter by seeing everyone’s patterns, but the actual packets and timestamps stay safely locked behind the local firewall.

C. Smart, Graded Throttling with RL

Our current binary throttle is a bit of a blunt instrument—it’s either on or off. That works for a lab, but it’s not ideal for a smooth user experience. We want a system that doesn’t just “cut the tap” but “adjusts the valve.”

By using Reinforcement Learning (RL), specifically an actor-critic setup, we could let the system learn a continuous range of actions. An RL agent could experiment with cutting the rate by 12%, 28%, or 45% based on the predicted severity of the congestion. It learns a reward function that balances two conflicting goals: keeping the queue empty while keeping the throughput as high as possible. Over time, it learns the “sweet spot” for different types of traffic without ever needing a human to label a single data point.

D. Spotting Outliers with Autoencoders

Standard classifiers are great at spotting things they’ve seen before, but they are “blind” to new types of problems. If a new type of IoT device starts pumping data in a way we didn’t anticipate, the MLP might just ignore it.

We can fix this by adding an Autoencoder as a “sanity check.” The Autoencoder is trained to perfectly reconstruct “healthy” network traffic. If the network starts acting weird, the Autoencoder will fail to reconstruct the telemetry accurately. This “reconstruction error” acts as a built-in alarm bell [10]. It lets us catch novel congestion signatures that weren’t in our original training set, giving the ACCF an extra layer of situational awareness.

E. Building Trust: Digital Twins and XAI

Network operators are naturally skeptical of “black box” AI. If the system throttles a critical medical stream, they need to know **why**. To solve this, we propose two things. First, Explainable AI (XAI) tools like SHAP. These tools break down a decision into human terms: “Throttled because latency jumped 20% while bandwidth dropped below 1 Mbps.”

Second, we can use a Digital Twin—a real-time virtual clone of the physical network. This lets us “test-drive” a throttle decision in the virtual world a split-second before we commit to it in the real world. If the Digital Twin predicts that a throttle will cause a crash or a timeout, the system can abort the action. It’s a “look before you leap” approach for 6G management.

ACKNOWLEDGEMENTS

A huge thanks goes out to the Mininet-WiFi development team at UNICAMP, Brazil. Their open-source platform was the backbone of this entire project and made this kind of experimentation possible. We also want to thank the IT Department at Netaji Subhas University of Technology (NSUT), Delhi, for providing the environment and support to dive into this research. Finally, credit is due to the Scikit-learn community; their tools allowed us to go from a whiteboard sketch to a working predictive model in a very short time.

REFERENCES

- [1] C. De Alwis, A. Kalla, Q. V. Pham, P. Kumar, K. Dev, W. J. Hwang, and M. Liyanage, "Survey on 6G frontiers: Trends, applications, requirements, technologies and future research," *IEEE Open Journal of the Communications Society*, vol. 2, pp. 836–886, 2021.
- [2] F. Guo, F. R. Yu, H. Zhang, X. Li, H. Ji, and V. C. M. Leung, "Enabling massive IoT toward 6G: A comprehensive survey," *IEEE Internet of Things Journal*, vol. 8, no. 15, pp. 11 891–11 915, 2021.
- [3] K. Nichols and V. Jacobson, "Controlling queue delay," *ACM Queue*, vol. 10, no. 5, pp. 20–34, May 2012.
- [4] S. Floyd and V. Jacobson, "Random early detection gateways for congestion avoidance," *IEEE/ACM Transactions on Networking*, vol. 1, no. 4, pp. 397–413, 1993.
- [5] D. Kreutz, F. M. V. Ramos, P. E. Verissimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.
- [6] R. R. Fontes, S. Afzal, S. H. B. Brito, M. A. S. Santos, and C. E. Rothenberg, "Mininet-WiFi: Emulating software-defined wireless networks," in *Proc. 11th Int. Conf. on Network and Service Management (CNSM)*. Barcelona, Spain: IEEE, 2015, pp. 384–389.
- [7] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proc. 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets-IX)*. ACM, 2010.
- [8] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <http://www.jmlr.org/papers/v12/pedregosa11a.html>
- [9] M. Reza, M. Javad, S. Raouf, and R. Javidan, "Network traffic classification using machine learning techniques over software defined networks," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 8, no. 7, 2017.
- [10] R. H. Serag *et al.*, "Machine-learning-based traffic classification in software-defined networks," *Electronics*, vol. 13, no. 6, p. 1108, 2024.